

개인 포트폴리오

BanG Dream!

채보 시스템을 분석해 재현한 리듬 게임 프로토타입



박 찬 욱

1. 개요

모바일 리듬 게임 **BanG Dream! (뱅드림)**의 채보 시스템을 코드 레벨에서 분석해 Unity 로 재현한 프로토타입이다. 리듬 게임의 핵심은 **노트가 정확한 박자에 판정선에 닿는 것**이고, 여기에 롱노트 · 슬라이드처럼 두 노트가 하나의 동작으로 이어지는 구조가 얹힌다. 이 두 가지를 어떻게 풀어내는지가 이 작품의 중심이었다. **1.5일** 만에 만들었다.

개발 환경	Unity · C# · 1인 개발
개발 기간	1.5일
소스 규모	스크립트 8개 / 약 850줄
구현 범위	텍스트 채보 파싱, 노트 6계열 13타입, 롱 · 슬라이드 연결, BPM 변경, 이펙트, 콤보

구현 기능

- 텍스트 채보 파일을 직접 파싱해 노트 큐로 만드는 재생 구조
- 실제 게임과 동일한 노트 타입 체계 (일반 · 플릭 · 롱 · 슬라이드 A/B · BPM 변경)
- 이동 시간만큼 앞서 생성하고 레인 길이에서 속도를 역산하는 타이밍 처리
- 롱 · 슬라이드 노트의 두 단계 연결과, 매 프레임 갱신되는 연결 빔
- 슬라이드가 다른 레인으로 넘어갈 때 판정 지점 자체가 이동하는 처리

2. 텍스트 채보 파싱

채보는 외부 텍스트 파일로 두고 실행 시 읽어 들인다. 파일 앞부분에 BPM 이 있고, 그 뒤로 한 줄에 노트 하나씩 **박자 / 노트타입 / 레인번호** 형식으로 나열된다. 노트를 코드에 하드코딩하지 않았기 때문에 곡을 바꾸려면 파일만 갈아끼우면 된다.

그림 1. 텍스트 채보 파일을 직접 파싱해 노트 큐를 만든다



원작과 동일한 노트 타입 체계를 그대로 지원한다

타입 번호	의미
1	일반
2	플릭
20	BPM 변경
21 · 25 · 26	롱노트 시작 · 끝 · 끝(플릭)
4 · 5 · 12	슬라이드 A 중간 · 끝 · 끝(플릭)
7 · 8 · 13	슬라이드 B 중간 · 끝 · 끝(플릭)

스킬 노트(11 · 31)는 연출만 다를 뿐 판정이 같아, 파싱 단계에서 기본 타입으로 바꿔 이후 처리를 단순화했다.

파싱은 `Split` 을 쓰지 않고 문자 커서를 직접 옮기며 `'/'` 를 만날 때마다 필드를 끊는 방식으로 구현했다. 읽기는 `ReadLineAsync` 로 비동기 처리해 곡 로딩이 프레임을 멈추지 않게 했다.

```
while (true)
{
    if ( cursor >= text.Length ) { ... 마지막 필드는 라인 번호 ... break; }
    else if ( text[cursor] == '/' )
    {
        if ( cnt == 0 ) beat = float.Parse( parsing ); // 박자
        if ( cnt == 1 ) noteType = (NoteType)int.Parse( parsing ); // 노트 타입
        ++cnt; ++cursor; parsing = "";
    }
    else { parsing += text[cursor]; ++cursor; } // 문자 모으기
}
```

스킬 노트(11·31)는 연출만 다를 뿐 판정 방식이 일반·롱 시작과 같다. 그래서 **파싱 단계에서 기본 타입으로 바꿔** 이후 로직이 다를 경우의 수를 줄였다. BPM 변경(20)도 별도의 이벤트 채널을 만들지 않고 **노트 스트림 안의 특수 타입**으로 끼워 넣어, 생성 시점에 BPM 값만 갈아끼우고 노트는 만들지 않는 방식으로 같은 파이프라인에서 처리했다.

3. 노트를 정확한 박자에 도착시키기

리듬 게임에서 노트는 **생성되는 순간이 아니라 판정선에 닿는 순간이** 박자와 맞아야 한다. 노트가 레인을 내려오는 데 시간이 걸리므로, 목표 박자에서 그 이동 시간만큼 **앞서서 생성**해야 한다. 이동 시간은 초 단위이고 박자는 BPM 기준이니, 초를 박자로 환산해서 빼준다.

그림 2. 노트가 정확한 박자에 판정선에 닿도록, 이동 시간만큼 앞서 생성한다



생성 시점 = 목표 박자 - (이동 시간 × BPM / 60)
`beatOfSpawn -= _config.goalTime * ( _config.bpm / 60f );`
`beatOfSpawn -= _config.customTime * ( _config.bpm / 60f );`

속도는 레인 길이에서 역산한다
`velocity = lane.dir * ( lane.dist / goalTime )` — 레인 길이가 달라도 이동 시간은 항상 같다

박자 시계는 `Main.Update` 에서 매 프레임 `bpm / 60 * deltaTime` 만큼 누적되는 값 하나로 관리한다. 생성기는 큐 맨 앞 노트의 생성 박자를 계산해 현재 박자가 그 지점을 넘었는지만 보면 된다.

```
// 목표 박자에서 이동 시간(초)을 박자로 환산해 빼면 생성 시점이 나온다
var beatOfSpawn = spawnInfo.beat;
beatOfSpawn -= _config.goalTime * ( _config.bpm / 60f );
beatOfSpawn -= _config.customTime * ( _config.bpm / 60f ); // 체감 보정값

if ( Main.instance.currBeat >= beatOfSpawn )
{
    _spawnInfoQueue.RemoveAt( 0 );
    SpawnNote( spawnInfo );
}

// 속도는 레인 길이에서 역산 — 레인 길이가 달라도 이동 시간은 goalTime 으로 같다
var velocity = lane.dir * ( lane.dist / Mathf.Max( 0.1f, _config.goalTime ) );
```

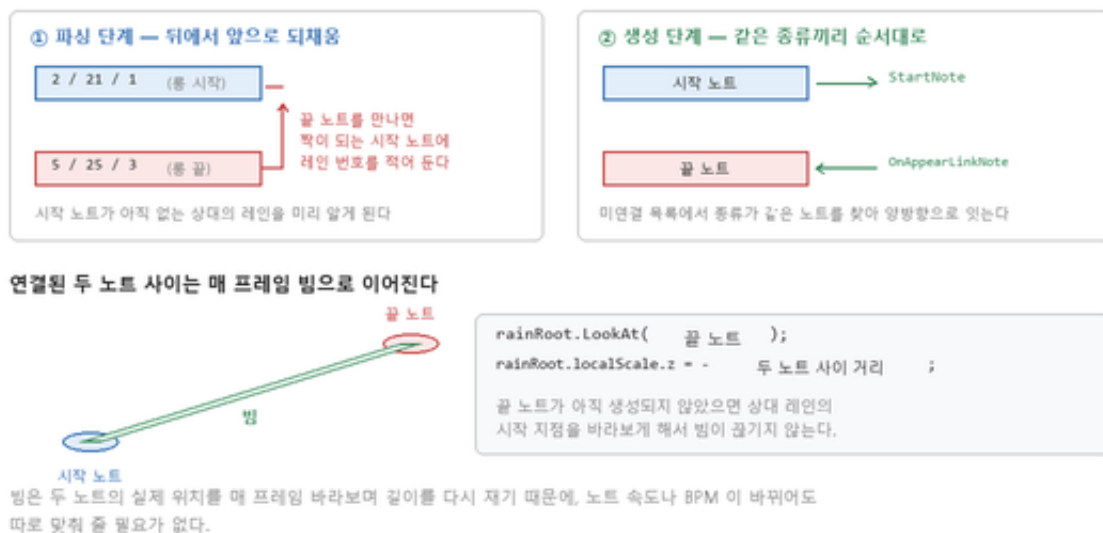
이 방식으로 얻은 것

- 속도를 직접 정하지 않는다. 레인 길이를 이동 시간으로 나눠 속도를 얻으므로, 원근 때문에 레인 길이가 서로 달라도 모든 노트가 똑같이 `goalTime` 초 뒤에 판정선에 닿는다.
- BPM 이 바뀌어도 식이 그대로다. 생성 시점을 박자로 계산하기 때문에 곡 중간에 BPM 이 바뀌면 환산 결과가 자동으로 따라간다.
- `customTime` 을 따로 뒤서, 기기나 사운드 지연에 따른 체감 어긋남을 코드 수정 없이 인스펙터에서 미세 조정할 수 있게 했다.

4. 롱 · 슬라이드 노트 연결

롱노트와 슬라이드는 시작 노트와 끝 노트가 짝을 이뤄 하나의 동작이 된다. 문제는 채보 파일에서 두 노트가 서로 다른 줄에 떨어져 있고, 시작 노트가 생성되는 시점에는 끝 노트가 아직 존재하지 않는다는 점이다. 그런데 시작 노트는 자기와 이어질 상대가 어느 레인에 있는지 알아야 연결 빔을 그 방향으로 뻗을 수 있다.

그림 3. 롱 · 슬라이드 노트를 두 단계로 짝지어 연결한다



그래서 연결을 두 단계로 나눴다. **파싱 단계**에서는 아직 짝을 못 찾은 시작 노트를 목록에 담아 두고, 끝 노트를 만나면 그 목록에서 짝을 찾아 **시작 노트 쪽에 상대의 레인 번호를 되채워** 준다. 덕분에 시작 노트는 생성될 때 이미 상대 레인을 알고 있다. **생성 단계**에서는 미연결 목록을 두고 같은 종류(롱 · 슬라이드 A · 슬라이드 B)끼리 순서대로 매칭해 두 노트를 양방향으로 잇는다.

```
// 생성 단계 — 미연결 목록에서 종류가 같은 노트를 찾아 잇는다
public void LinkPreviousNote( Note pushNote, List<Note> unlinks )
{
    foreach ( var unlink in unlinks )
    {
        if ( unlink.noteKind != pushNote.noteKind ) continue;

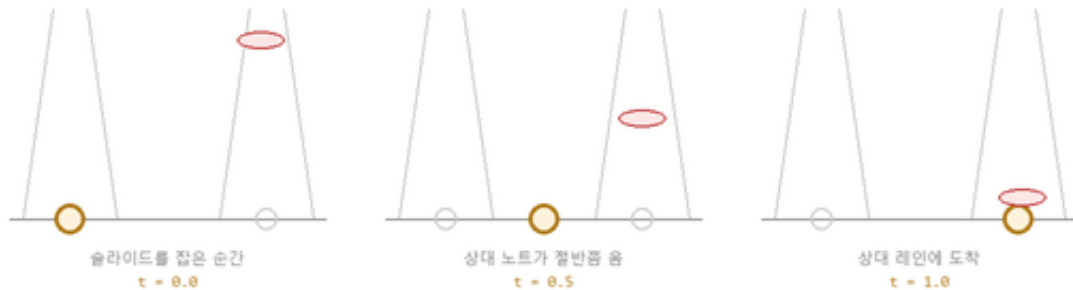
        pushNote.StartNote = unlink; // 끝 → 시작
        unlink.OnAppearLinkNote( pushNote ); // 시작 → 끝
        unlinks.Remove( unlink );
        break;
    }
}
```

연결된 두 노트 사이는 매 프레임 갱신되는 빔으로 이어진다. 빔은 상대 노트를 바라보게 회전시키고 길이를 두 노트 사이 거리로 다시 잡는 것이 전부다. 상대 노트가 아직 생성되지 않았다면 **상대 레인의 시작 지점**을 바라보게 해서, 화면 밖에서 이어져 들어오는 것처럼 빔이 끊기지 않는다. 실제 위치를 매 프레임 다시 보기 때문에 속도나 BPM 이 바뀌어도 따로 맞춰 줄 필요가 없다.

5. 슬라이드 — 판정 지점이 레인을 넘어간다

슬라이드 노트는 손가락이 한 레인에서 다른 레인으로 미끄러지는 동작이다. 이걸 노트만 옮겨서 표현하면 판정선과 어긋나 보인다. 그래서 **판정 지점 자체를 다음 레인 쪽으로 이동시키는** 방식을 썼다.

그림 4. 슬라이드는 판정 지점 자체를 다음 레인으로 옮긴다



진행도 t 는 '상대 노트가 자기 판정선까지 얼마나 남았는가'로 구현다

$t = 1 - (\text{상대 노트의 남은 거리} / \text{처음 잡았을 때의 남은 거리})$

판정 지점을 이 t 로 원래 자리에서 상대 레인 자리까지 선형 보간해 옮긴다. 손가락이 레인을 넘어가는 동작이 끝 판정 지점의 이동이 되고, 슬라이드가 끝나면 원래 자리로 되돌린다.

진행도는 **상대 노트가 자기 판정선까지 얼마나 남았는가**로 구한다. 슬라이드를 처음 잡았을 때의 남은 거리를 기준으로 두고, 현재 남은 거리와의 비율을 뒤집으면 0에서 1로 가는 값이 된다. 이 값으로 판정 지점을 원래 자리에서 상대 레인 자리까지 선형 보간한다.

```
private void Move()
{
    if ( !_hooked ) return;

    goal.position = Vector3.Lerp(
        originGoalPos, // 원래 판정 지점
        _hookedNote.linkLane.goal.position, // 상대 레인의 판정 지점
        1f - ( _hookedNote.linkNoteRemainDist / _linkNoteRemainingDistFirst ) );
}
```

판정은 거리 허용 범위가 아니라 **노트가 판정 지점을 지나쳤는가**로 본다. 판정 지점에서 노트를 향한 벡터의 z 성분 부호가 뒤집히는 순간이 통과 시점이다. 슬라이드가 끝나면 판정 지점을 원래 자리로 되돌리고 이펙트를 정리한다.

6. 이펙트와 전체 구조

타격 이펙트는 스프라이트 시트를 `FixedUpdate` 에서 한 장씩 넘기는 프레임 애니메이션이다. 일반 · 플릭 이펙트는 마지막 장에서 꺼지고, **슬라이드 이펙트는 처음으로 되돌아가 반복된다** — 슬라이드는 손가락을 떼기 전까지 계속 유지되는 상태이기 때문이다. 노트 스프라이트도 레인 번호로 골라서, 원작처럼 레인마다 색이 다르게 나온다.



노트가 흘러 내려오는 화면. 가운데 초록색 빔이 연결된 롱 · 슬라이드 노트, 아래쪽 원형 이펙트가 타격 연출.

역할 분담

Main	채보 파일 로드, 박자 시계 누적, 콤보 집계
NoteSpawner	채보 파싱, 생성 시점 계산, 노트 생성과 연결
Note	이동, 연결 빔 갱신, 자기 타입에 따른 표시 전환
Lane	판정 지점 관리, 통과 판정과 타입별 처리, 슬라이드 시 판정 지점 이동
Config	BPM · 이동 시간 · 보정값 — 인스펙터에서 조정하는 값 모음
Effect	타격 이펙트 프레임 애니메이션

회고

리듬 게임에서 배운 가장 큰 것은 **기준을 시간이 아니라 박자로 잡아야 한다는** 것이었다. 처음에는 초 단위로 타이밍을 맞추려 했는데 BPM 이 바뀌는 순간 전부 어긋났다. 박자를 기준 축으로 두고 필요할 때만 초로 환산하는 구조로 바꾸고 나서야 BPM 변경이 자연스럽게 흡수됐다.

연결 노트에서는 **아직 존재하지 않는 대상을 가리켜야 하는 문제**를 파싱 단계에서 미리 정보를 되채우는 방식으로 풀었다. 런타임에 해결하려 했다면 훨씬 복잡했을 것을, 데이터를 읽는 시점에 한 번 정리해 두는 것으로 단순하게 만들 수 있었다.

다만 1.5일 만에 만든 프로토타입이라 남은 부분도 많다. **입력 판정이 없어** 노트가 판정선을 지나가면 자동으로 처리되고, 따라서 Perfect · Great 같은 정확도 등급과 미스 처리가 없다. 노트를 매번 생성 · 파기하는 것도 실제 곡 하나를 돌리기에는 부담이라 오브젝트 풀이 필요하다.